



Работа С Arduino И MPU6050

Arduino и датчик MPU6050
Подключение
Получение сырых данных
Библиотека MPU6050 от i2cdev
Расшифровка сырых данных
Настройка чувствительности
Калибровка оффсетов
Алгоритмы калибровки
Получение углов
Оптимальное получение углов
Некоторые примеры
ПОХОЖИЕ ЗАПИСИ

Arduino и датчик MPU6050

Статья обновлена



Добавил всякой документации, исправил старые примеры и добавил новые. Спасибо за активность!

MPU6050 представляет собой 3-х осевой гироскоп и 3-х же осевой акселерометр в одном корпусе. Сразу поясню, что это и для чего:

- Гироскоп измеряет **угловую скорость вращения вокруг оси**, условно в градусах/секунду. Если датчик лежит на столе – по всем трём осям будет значение около нуля. Для нахождения текущего угла по скорости нужно интегрировать эту скорость. Гироскоп может чуть привирать, поэтому ориентироваться на него для расчёта текущего угла не получится даже при идеальной калибровке.
- Акселерометр измеряет **ускорение вдоль оси**, условно в метрах/секунду/секунду. Если датчик лежит на столе или движется с постоянной скоростью – на оси будет спроецирован вектор силы тяжести. Если датчик движется с ускорением – вдобавок к ускорению свободного падения получим составляющие вектора ускорения. Если датчик находится в свободном падении (в том числе на орбите планеты) – величины ускорений по всем осям будут равны 0. Зная проекции вектора силы тяжести можно с высокой точностью определить угол наклона датчика относительно него (привет, школьная математика). Если датчик движется – однозначно определить направление вектора силы тяжести не получится, соответственно угол тоже.

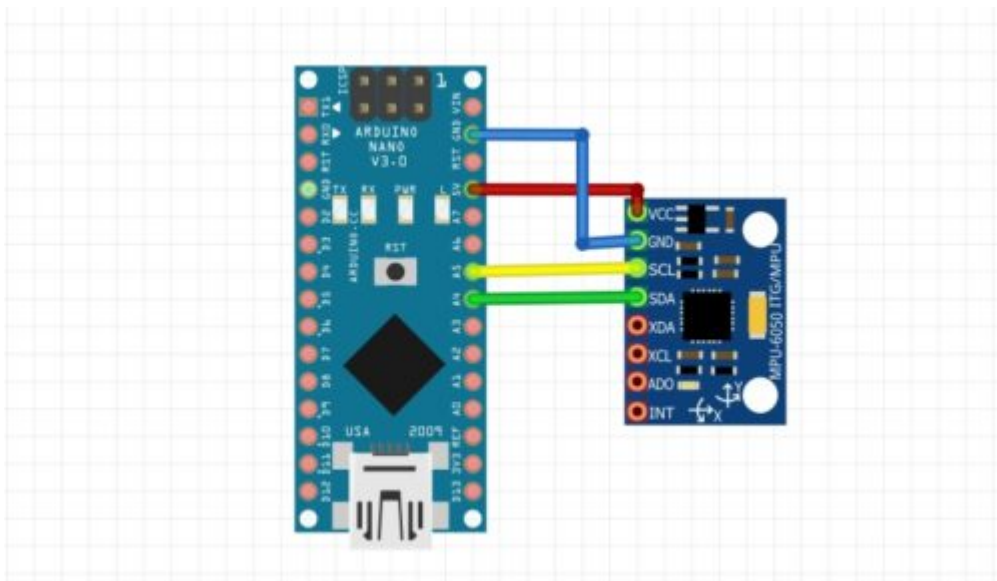


Модуль стоит в районе 150 рублей на нашем любимом AliExpress ([ссылка](#), [ссылка](#)), а также входит в [Arduino набор GyverKIT](#).

Итак, по отдельности акселерометр и гироскоп не могут обеспечить точное измерение угла, но вместе – ещё как могут! Об этом мы поговорим ниже. А начнём с подключения и базового общения с датчиком.

Подключение

Датчик подключается на шину i2c (**SDA -> A4**, **SCL -> A5**, **GND -> GND**). На плате стоит стабилизатор, позволяющий питаться от пина 5V (**VCC -> 5V**).



На модуле выведен пин **AD0**. Если он никуда не подключен (или подключен к **GND**) – адрес датчика на шине i2c будет 0x68, если подключен к питанию (**VCC**) – адрес будет 0x69. Таким образом без дополнительных микросхем можно подключить два датчика на шину с разными адресами.

Получение сырых данных

С MPU6050 можно общаться напрямую при помощи встроенной библиотеки *Wire.h* и даташита, а можно взять очень мощную библиотеку от i2cdev: [официальный сайт](#) (на момент написания не работает) или [GitHub](#). Для работы понадобится сама библиотека MPU6050 и библиотека I2Cdev (удобного способа скачать там нет, поэтому выложил архив с обеими на [Яндекс.Диск](#)). Мы будем работать с библиотекой, но я оставлю пример опроса всех ускорений и угловых скоростей напрямую, т.к. библиотека тяжёлая, а пример очень лёгкий.

↓ Опрос вручную по i2c

```
include "Wire.h"
int MPU_addr = 0x68; // адрес датчика
```

```
// массив данных
// [accX, accY, accZ, temp, gyrX, gyrY, gyrZ]
// acc - ускорение, gyr - угловая скорость, temp - температура (raw)
int16_t data[7];
void setup() {
    // инициализация
    Wire.begin();
    Wire.beginTransmission(MPU_addr);
    Wire.write(0x6B); // PWR_MGMT_1 register
    Wire.write(0);    // set to zero (wakes up the MPU-6050)
    Wire.endTransmission(true);

    Serial.begin(9600);
}
void loop() {
    getData(); // получаем
    // выводим
    for (byte i = 0; i < 7; i++) {
        Serial.print(data[i]);
        Serial.print('\t');
    }
    Serial.println();
    delay(200);
}
void getData() {
    Wire.beginTransmission(MPU_addr);
    Wire.write(0x3B); // starting with register 0x3B (ACCEL_XOUT_H)
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_addr, 14, true); // request a total of 14 registers
    for (byte i = 0; i < 7; i++) {
        data[i] = Wire.read() << 8 | Wire.read();
    }
}
```

↓ Опрос через библиотеку i2cdev

```
#include "MPU6050.h"
MPU6050 mpu;
int16_t ax, ay, az;
float gx, gy, gz;

void setup() {
```

```
Wire.begin();
Serial.begin(9600);
mpu.initialize();
// состояние соединения
Serial.println(mpu.testConnection() ? "MPU6050 OK" : "MPU6050 FAIL");
delay(1000);
}
void loop() {
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print('\t');
  Serial.print(ay); Serial.print('\t');
  Serial.print(az); Serial.print('\t');
  Serial.print(gx); Serial.print('\t');
  Serial.print(gy); Serial.print('\t');
  Serial.println(gz);
  delay(5);
}
```

Библиотека MPU6050 от i2cdev

Эта библиотека является одной из самых сложных в Ардуино-среде, да и сам датчик чего стоит – 50 страниц даташита + 50 страниц с описанием регистров и настроек... А в библиотеке – несколько сотен (!) функций для работы с модулем и его многочисленными настройками. Рассмотрим самые “нужные”, которые точно пригодятся в работе:

↓ Методы для MPU6050

```
// ===== ГИРОСКОП =====
// получить угловые скорости по осям
int16_t getRotationX();
int16_t getRotationY();
int16_t getRotationZ();
void getRotation(int16_t* x, int16_t* y, int16_t* z);

// получить диапазон гироскопа
// 0 = +/- 250 degrees/sec
// 1 = +/- 500 degrees/sec
// 2 = +/- 1000 degrees/sec
// 3 = +/- 2000 degrees/sec
uint8_t getFullScaleGyroRange();

// установить диапазон гироскопа в формате
```

```
// MPU6050_GYRO_FS_250
// MPU6050_GYRO_FS_500
// MPU6050_GYRO_FS_1000
// MPU6050_GYRO_FS_2000
void setFullScaleGyroRange(uint8_t range);

// ===== АКСЕЛЕРОМЕТР =====
// получить ускорения по осям
int16_t getAccelerationX();
int16_t getAccelerationY();
int16_t getAccelerationZ();
void getAcceleration(int16_t* x, int16_t* y, int16_t* z);

// получить диапазон акселя
// 0 = +/- 2g
// 1 = +/- 4g
// 2 = +/- 8g
// 3 = +/- 16g
uint8_t getFullScaleAccelRange();

// установить диапазон акселя в формате
// MPU6050_ACCEL_FS_2
// MPU6050_ACCEL_FS_4
// MPU6050_ACCEL_FS_8
// MPU6050_ACCEL_FS_16
void setFullScaleAccelRange(uint8_t range);

// ===== ВСЯКОЕ =====
// получить ускорение, угловую скорость
void getMotion6(int16_t* ax, int16_t* ay, int16_t* az, int16_t* gx, int16_t* gy

// получить температуру
int16_t getTemperature();

// перезагрузить
void reset();

// управление режимом сна
  getSleepEnabled();
 setSleepEnabled(bool enabled);
```

```
// ===== ПЕРЫВАНИЕ СВОБОДНОГО ПАДЕНИЯ =====
// порог датчика свободного падения 0..255. Одна единица - 2mg
uint8_t getFreefallDetectionThreshold();           // получить порог
void setFreefallDetectionThreshold(uint8_t threshold); // установить порог

// время определения свободного падения 0..255. Одна единица - 1мс
uint8_t getFreefallDetectionDuration();           // получить время
void setFreefallDetectionDuration(uint8_t duration); // установить время

// прерывание
bool getIntFreefallEnabled();
void setIntFreefallEnabled(bool enabled);

// ===== ПЕРЫВАНИЕ ДВИЖЕНИЯ =====
// порог датчика движения 0..255. Одна единица - 2mg
uint8_t getMotionDetectionThreshold();           // получить порог
void setMotionDetectionThreshold(uint8_t threshold); // установить порог

// время определения движения 0..255. Одна единица - 1мс
uint8_t getMotionDetectionDuration();           // получить время
void setMotionDetectionDuration(uint8_t duration); // установить время

// прерывание
bool getIntMotionEnabled();
void setIntMotionEnabled(bool enabled);

// ===== ПЕРЫВАНИЕ ОТСУТСТВИЯ ДВИЖЕНИЯ =====
// порог определения отсутствия движения 0..255. Одна единица - 2mg
uint8_t getZeroMotionDetectionThreshold();           // получить порог
void setZeroMotionDetectionThreshold(uint8_t threshold); // установить порог

// время определения отсутствия движения 0..255. Одна единица - 1мс
uint8_t getZeroMotionDetectionDuration();           // получить время
void setZeroMotionDetectionDuration(uint8_t duration); // установить время

// прерывание
bool getIntZeroMotionEnabled();
void setIntZeroMotionEnabled(bool enabled);
```



```
// ===== ПЕРЕРЫВАНИЯ =====  
// включить/выключить прерывания  
bool getInterruptMode();  
void setInterruptMode(bool mode);  
  
// байт со всеми прерываниями. Настройка прерываний  
uint8_t getIntEnabled();  
void setIntEnabled(uint8_t enabled);  
  
// статусы прерываний  
uint8_t getIntStatus();  
bool getIntFreefallStatus();  
bool getIntMotionStatus();  
bool getIntZeroMotionStatus();  
  
// ===== ОФФСЕТЫ =====  
// установить. XYZ аксель, XYZ гироскоп  
void setXAccelOffset(int16_t offset);  
void setYAccelOffset(int16_t offset);  
void setZAccelOffset(int16_t offset);  
void setXGyroOffset(int16_t offset);  
void setYGyroOffset(int16_t offset);  
void setZGyroOffset(int16_t offset);  
  
// получить. XYZ аксель, XYZ гироскоп  
int16_t getXAccelOffset();  
int16_t getYAccelOffset();  
int16_t getZAccelOffset();  
int16_t getXGyroOffset();  
int16_t getYGyroOffset();  
int16_t getZGyroOffset();  
  
// вывести оффсеты в порт  
void PrintActiveOffsets();  
  
void CalibrateGyro(uint8_t Loops = 15);    // калибровать гироскоп(кол-во итераций)  
void CalibrateAccel(uint8_t Loops = 15);    // калибровать акселерометр(кол-во итераций)
```



По каждой оси и параметру датчик выдаёт 16-битное знаковое значение (`-32768.. 32767`).
При стандартных настройках (как в примерах выше) это значение отражает:

- Ускорение в диапазоне `-2.. 2 g` (`9.82 м/с/с`).
- Угловую скорость в диапазоне `-250.. 250` градусов/секунду.

Таким образом, для перевода сырых данных в величины СИ (если это нужно) можно сделать по каждой оси:

- Ускорение в м/с/с при чувствительности 2: `float accX_f = accX / 32768 * 2`
- Угловая скорость в град/с при чувствительности 250: `float gyrX_f = gyrX / 32768 * 250`

Думаю закономерность понятна.

Настройка чувствительности

Датчик позволяет настроить чувствительность по каждому параметру, в библиотеке `i2cdev` это делается так:

- `setFullScaleAccelRange(MPU6050_ACCEL_FS_2);` – диапазон `-2.. 2 g`
- `setFullScaleAccelRange(MPU6050_ACCEL_FS_4);` – диапазон `-4.. 4 g`
- `setFullScaleAccelRange(MPU6050_ACCEL_FS_8);` – диапазон `-8.. 8 g`
- `setFullScaleAccelRange(MPU6050_ACCEL_FS_16);` – диапазон `-16.. 16 g`
- `setFullScaleGyroRange(MPU6050_GYRO_FS_250);` – диапазон `-250.. 250 град/с`
- `setFullScaleGyroRange(MPU6050_GYRO_FS_500);` – диапазон `-500.. 500 град/с`
- `setFullScaleGyroRange(MPU6050_GYRO_FS_1000);` – диапазон `-1000.. 1000 град/с`
- `setFullScaleGyroRange(MPU6050_GYRO_FS_2000);` – диапазон `-2000.. 2000 град/с`

Ну и пример

```
void setup() {
  Wire.begin();
  mpu.initialize();
  mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_4);
  mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_1000);
}
```

Перевести сырые данные можно таким же способом, как в предыдущем пункте, но с учётом нового диапазона.

Калибровка оффсетов

Для достижения максимальной точности измерений нужно откалибровать акселерометр и гироскоп. Калибровка акселерометра позволяет выставить “ноль” для вектора силы тяжести, а калибровка гироскопа уменьшает его “дрифт”, то есть статическое отклонение в режиме покоя.

Идеально откалиброванный и лежащий горизонтально датчик должен показывать ускорение ~16384 по оси Z и нули по всем остальным осям ускорения и угловой скорости. Но это фантастика =)

Максимально правильно использовать калибровку в проекте нужно так: калибровка по запросу (кнопка, меню, и т.д.), затем запись калибровочных значений в EEPROM. При запуске – чтение и настройка оффсетов из. Рассмотрим несколько примеров калибровки, первый – из библиотеки. Калибрует долго, но максимально точно. *При малых вибрациях и движениях датчика в процессе калибровки (даже от громкого звука) калибровка может не закончиться.* Второй вариант – мой упрощённый алгоритм калибровки, калибрует быстро, без возможности зависнуть при тряске, но даёт менее точный результат. Я делюсь примерами, в свой проект их нужно будет переносить вручную и аккуратно =)

Алгоритмы калибровки

Для достижения максимальной точности измерений нужно откалибровать акселерометр и гироскоп. Калибровка акселерометра позволяет выставить “ноль” для вектора силы тяжести, а калибровка гироскопа уменьшает его “дрифт”, то есть статическое отклонение в режиме покоя. Идеально откалиброванный и лежащий горизонтально датчик должен показывать ускорение ~16384 по оси Z и нули по всем остальным осям ускорения и угловой скорости. Но это фантастика =) Максимально правильно использовать калибровку в проекте нужно так: калибровка по запросу (кнопка, меню, и т.д.), затем запись калибровочных значений в EEPROM. При запуске – чтение и настройка оффсетов. Да, можно замерить значения по всем 6 осям в покое, сохранить их в переменные, а затем вычитать из свежих прочитанных в процессе работы. Такой способ работает для каких-то базовых операций с датчиком (определение перегрузок, тряски, наличия вращения, и т.д.). Для использования MPU6050 с целью максимально точного определения углов поворота платы такой вариант к сожалению не подходит: калибровать нужно *рекурсивно*. Рассмотрим несколько примеров калибровки, первый – из библиотеки. Калибрует долго, но максимально точно. *При малых вибрациях и движениях датчика в процессе калибровки (даже от громкого звука) калибровка может не закончиться.* Второй вариант – мой упрощённый алгоритм калибровки, калибрует быстро, без возможности зависнуть при тряске, но даёт менее точный результат. Я делюсь примерами, в свой проект их нужно будет переносить вручную и аккуратно =)

↓ Калибровка из примера библиотеки

```
// положи датчик горизонтально, надписями вверх
// ДАЖЕ НЕ ДЫШИ НА НЕГО
// отправь любой символ в сериал, чтобы начать калибровку
// жди результатов

// ----- НАСТРОЙКИ -----
t int buffersize = 70;    // количество итераций калибровки
: int accel_deadzone = 10; // точность калибровки акселерометра (по умолчанию 2
const int gyro_deadzone = 6; // точность калибровки гироскопа (по умолчанию 2
```

```
// ----- НАСТРОЙКИ -----

#include "I2Cdev.h"
#include "MPU6050.h"
#include "Wire.h"
MPU6050 mpu(0x68);
int16_t ax, ay, az, gx, gy, gz;
int mean_ax, mean_ay, mean_az, mean_gx, mean_gy, mean_gz, state = 0;
int ax_offset, ay_offset, az_offset, gx_offset, gy_offset, gz_offset;

//////////////////////////////////// SETUP //////////////////////////////////////
void setup() {
    Wire.begin();
    Serial.begin(9600);
    mpu.initialize();
    // ждём очистки сериал
    while (Serial.available() && Serial.read()); // чистим
    while (!Serial.available()) {
        Serial.println(F("Send any character to start sketch.\n"));
        delay(1500);
    }
    while (Serial.available() && Serial.read()); // чистим ещё на всякий
    Serial.println("\nMPU6050 Calibration Sketch");
    delay(2000);
    Serial.println("\nYour MPU6050 should be placed in horizontal position, with");
    delay(3000);

    // проверка соединения
    Serial.println(mpu.testConnection() ? "MPU6050 connection successful" : "MPU6050 connection failed");
    delay(1000);

    // сбросить оффсеты
    mpu.setXAccelOffset(0);
    mpu.setYAccelOffset(0);
    mpu.setZAccelOffset(0);
    mpu.setXGyroOffset(0);
    mpu.setYGyroOffset(0);
    mpu.setZGyroOffset(0);
}

//////////////////////////////////// LOOP //////////////////////////////////////
loop() {
    if (state == 0) {
```

```
    Serial.println("\nReading sensors for first time...");
    meansensors();
    state++;
    delay(1000);
}
if (state == 1) {
    Serial.println("\nCalculating offsets...");
    calibration();
    state++;
    delay(1000);
}
if (state == 2) {
    meansensors();
    Serial.println("\nFINISHED!");
    Serial.print("\nSensor readings with offsets:\t");
    Serial.print(mean_ax);
    Serial.print("\t");
    Serial.print(mean_ay);
    Serial.print("\t");
    Serial.print(mean_az);
    Serial.print("\t");
    Serial.print(mean_gx);
    Serial.print("\t");
    Serial.print(mean_gy);
    Serial.print("\t");
    Serial.println(mean_gz);
    Serial.print("Your offsets:\t");
    Serial.print(ax_offset);
    Serial.print(", ");
    Serial.print(ay_offset);
    Serial.print(", ");
    Serial.print(az_offset);
    Serial.print(", ");
    Serial.print(gx_offset);
    Serial.print(", ");
    Serial.print(gy_offset);
    Serial.print(", ");
    Serial.println(gz_offset);
    Serial.println("\nData is printed as: accelX accelY accelZ giroX giroY giroZ");
    Serial.println("Check that your sensor readings are close to 0 0 16384 0 0 0");
    Serial.println("If calibration was succesful write down your offsets so you");
    while (1);
}
```



```

}

//////////////////////////////////////  FUNCTIONS  ////////////////////////////////////////

void meansensors() {
    long i = 0, buff_ax = 0, buff_ay = 0, buff_az = 0, buff_gx = 0, buff_gy = 0, buff_gz = 0;
    while (i < (buffersize + 101)) { // read raw accel/gyro measurements from device
        mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
        if (i > 100 && i <= (buffersize + 100)) { //First 100 measures are discarded
            buff_ax = buff_ax + ax;
            buff_ay = buff_ay + ay;
            buff_az = buff_az + az;
            buff_gx = buff_gx + gx;
            buff_gy = buff_gy + gy;
            buff_gz = buff_gz + gz;
        }
        if (i == (buffersize + 100)) {
            mean_ax = buff_ax / buffersize;
            mean_ay = buff_ay / buffersize;
            mean_az = buff_az / buffersize;
            mean_gx = buff_gx / buffersize;
            mean_gy = buff_gy / buffersize;
            mean_gz = buff_gz / buffersize;
        }
        i++;
        delay(2);
    }
}

void calibration() {
    ax_offset = -mean_ax / 8;
    ay_offset = -mean_ay / 8;
    az_offset = (16384 - mean_az) / 8;
    gx_offset = -mean_gx / 4;
    gy_offset = -mean_gy / 4;
    gz_offset = -mean_gz / 4;

    while (1) {
        int ready = 0;
        mpu.setXAccelOffset(ax_offset);
        mpu.setYAccelOffset(ay_offset);
        mpu.setZAccelOffset(az_offset);
        mpu.setXGyroOffset(gx_offset);
        mpu.setYGyroOffset(gy_offset);
        mpu.setZGyroOffset(gz_offset);
    }
}

```



```

meansensors();
Serial.println("...");

if (abs(mean_ax) <= acel_deadzone) ready++;
else ax_offset = ax_offset - mean_ax / acel_deadzone;
if (abs(mean_ay) <= acel_deadzone) ready++;
else ay_offset = ay_offset - mean_ay / acel_deadzone;
if (abs(16384 - mean_az) <= acel_deadzone) ready++;
else az_offset = az_offset + (16384 - mean_az) / acel_deadzone;
if (abs(mean_gx) <= gyro_deadzone) ready++;
else gx_offset = gx_offset - mean_gx / (gyro_deadzone + 1);
if (abs(mean_gy) <= gyro_deadzone) ready++;
else gy_offset = gy_offset - mean_gy / (gyro_deadzone + 1);
if (abs(mean_gz) <= gyro_deadzone) ready++;
else gz_offset = gz_offset - mean_gz / (gyro_deadzone + 1);
if (ready == 6) break;
}
}

```

↓ Мой вариант калибровки

```

#include "Wire.h"
#include "I2Cdev.h"
#include "MPU6050.h"
MPU6050 mpu;
#define BUFFER_SIZE 100
int16_t ax, ay, az;
int16_t gx, gy, gz;

void setup() {
  Wire.begin();
  Serial.begin(9600);
  mpu.initialize();
  mpu.setXAccelOffset(0);
  mpu.setYAccelOffset(0);
  mpu.setZAccelOffset(0);
  mpu.setXGyroOffset(0);
  mpu.setYGyroOffset(0);
  mpu.setZGyroOffset(0);
  Serial.println(F("Send any character to start sketch"));
}

```

```
delay(100);
while (1) { //входим в бесконечный цикл
    if (Serial.available() > 0) { //если нажата любая кнопка
        Serial.read(); //прочитать (чтобы не висел в буфере)
        break; //выйти из цикла
    }
}
delay(1000);
}

void loop() {
    // выводим начальные значения
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    Serial.print(ax); Serial.print(" ");
    Serial.print(ay); Serial.print(" ");
    Serial.print(az); Serial.print(" ");
    Serial.print(gx); Serial.print(" ");
    Serial.print(gy); Serial.print(" ");
    Serial.println(gz);
    calibration();

    // выводим значения после калибровки
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    Serial.print(ax); Serial.print(" ");
    Serial.print(ay); Serial.print(" ");
    Serial.print(az); Serial.print(" ");
    Serial.print(gx); Serial.print(" ");
    Serial.print(gy); Serial.print(" ");
    Serial.println(gz);
    delay(20);
    while (1);
}

// ===== ФУНКЦИЯ КАЛИБРОВКИ =====
void calibration() {
    long offsets[6];
    long offsetsOld[6];
    int16_t mpuGet[6];

    // используем стандартную точность
    mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);
    mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_250);
```

```
// обнуляем оффсеты
mpu.setXAccelOffset(0);
mpu.setYAccelOffset(0);
mpu.setZAccelOffset(0);
mpu.setXGyroOffset(0);
mpu.setYGyroOffset(0);
mpu.setZGyroOffset(0);
delay(10);
Serial.println("Calibration start. It will take about 5 seconds");
for (byte n = 0; n < 10; n++) {    // 10 итераций калибровки
    for (byte j = 0; j < 6; j++) {    // обнуляем калибровочный массив
        offsets[j] = 0;
    }
    for (byte i = 0; i < 100 + BUFFER_SIZE; i++) { // делаем BUFFER_SIZE измере
        mpu.getMotion6(&mpuGet[0], &mpuGet[1], &mpuGet[2], &mpuGet[3], &mpuGet[4]
        if (i >= 99) {                // пропускаем первые 99 измерений
            for (byte j = 0; j < 6; j++) {
                offsets[j] += (long)mpuGet[j];    // записываем в калибровочный массив
            }
        }
    }
    for (byte i = 0; i < 6; i++) {
        offsets[i] = offsetsOld[i] - ((long)offsets[i] / BUFFER_SIZE); // учитыва
        if (i == 2) offsets[i] += 16384;                                // если ос
        offsetsOld[i] = offsets[i];
    }
    // ставим новые оффсеты
    mpu.setXAccelOffset(offsets[0] / 8);
    mpu.setYAccelOffset(offsets[1] / 8);
    mpu.setZAccelOffset(offsets[2] / 8);
    mpu.setXGyroOffset(offsets[3] / 4);
    mpu.setYGyroOffset(offsets[4] / 4);
    mpu.setZGyroOffset(offsets[5] / 4);
    delay(2);
}
/*
// выводим в порт
Serial.println("Calibration end. Your offsets:");
Serial.println("accX accY accZ gyrX gyrY gyrZ");
Serial.print(mpu.getXAccelOffset()); Serial.print(", ");
Serial.print(mpu.getYAccelOffset()); Serial.print(", ");
Serial.print(mpu.getZAccelOffset()); Serial.print(", ");
```



```

    Serial.print(mpu.getXGyroOffset()); Serial.print(", ");
    Serial.print(mpu.getYGyroOffset()); Serial.print(", ");
    Serial.print(mpu.getZGyroOffset()); Serial.println(" ");
    Serial.println(" ");
  */
}

```

↓ Мой вариант калибровки + запись в EEPROM

```

// калибровка и запись в EEPROM
// при запуске настраиваем оффсеты
// послать символ в сериал - начать повторную калибровку
#include "Wire.h"
#include "I2Cdev.h"
#include "MPU6050.h"

MPU6050 mpu;
#define BUFFER_SIZE 100
#define START_BYTE 1010
int16_t ax, ay, az;
int16_t gx, gy, gz;

void setup() {
  Wire.begin();
  Serial.begin(9600);
  mpu.initialize();

  //----- ВСПОМИНАЕМ ОФФСЕТЫ -----
  int offsets[6];
  EEPROM.get(START_BYTE, offsets);

  // ставим оффсеты из памяти
  mpu.setXAccelOffset(offsets[0]);
  mpu.setYAccelOffset(offsets[1]);
  mpu.setZAccelOffset(offsets[2]);
  mpu.setXGyroOffset(offsets[3]);
  mpu.setYGyroOffset(offsets[4]);
  mpu.setZGyroOffset(offsets[5]);

  //----- ВСПОМИНАЕМ ОФФСЕТЫ -----

```




```

Serial.println(F("Send any character to start sketch"));
delay(100);
while (1) {
    //входим в бесконечный цикл
    if (Serial.available() > 0) {    // если нажата любая кнопка
        Serial.read();              // прочитать (чтобы не висел в буфере)
        break;                      // выйти из цикла
    }
}
delay(1000);
}

void loop() {
    calibration();
    Serial.println("Current offsets:");
    Serial.println("accX accY accZ gyrX gyrY gyrZ");
    Serial.print(mpu.getXAccelOffset()); Serial.print(", ");
    Serial.print(mpu.getYAccelOffset()); Serial.print(", ");
    Serial.print(mpu.getZAccelOffset()); Serial.print(", ");
    Serial.print(mpu.getXGyroOffset()); Serial.print(", ");
    Serial.print(mpu.getYGyroOffset()); Serial.print(", ");
    Serial.print(mpu.getZGyroOffset()); Serial.println(" ");
    Serial.println(" ");

    // выводим значения после калибровки
    Serial.println("Readings:");
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    Serial.print(ax); Serial.print(" ");
    Serial.print(ay); Serial.print(" ");
    Serial.print(az); Serial.print(" ");
    Serial.print(gx); Serial.print(" ");
    Serial.print(gy); Serial.print(" ");
    Serial.println(gz);
    delay(20);
    while (1);
}

// ===== ФУНКЦИЯ КАЛИБРОВКИ И ЗАПИСИ В ЕЕПРОМ =====
void calibration() {
    long offsets[6];
    long offsetsOld[6];
    uint16_t mpuGet[6];

```

```
// используем стандартную точность
mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);
mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_250);

// обнуляем оффсеты
mpu.setXAccelOffset(0);
mpu.setYAccelOffset(0);
mpu.setZAccelOffset(0);
mpu.setXGyroOffset(0);
mpu.setYGyroOffset(0);
mpu.setZGyroOffset(0);
delay(5);

for (byte n = 0; n < 10; n++) {    // 10 итераций калибровки
    for (byte j = 0; j < 6; j++) {    // обнуляем калибровочный массив
        offsets[j] = 0;
    }
    for (byte i = 0; i < 100 + BUFFER_SIZE; i++) {
        // делаем BUFFER_SIZE измерений для усреднения
        mpu.getMotion6(&mpuGet[0], &mpuGet[1], &mpuGet[2], &mpuGet[3], &mpuGet[4])

        if (i >= 99) {                // пропускаем первые 99 измерений
            for (byte j = 0; j < 6; j++) {
                offsets[j] += (long)mpuGet[j];    // записываем в калибровочный массив
            }
        }
    }
}

for (byte i = 0; i < 6; i++) {
    offsets[i] = offsetsOld[i] - ((long)offsets[i] / BUFFER_SIZE); // учитыва
    if (i == 2) offsets[i] += 16384;                                // если ос
    offsetsOld[i] = offsets[i];
}

// ставим новые оффсеты
mpu.setXAccelOffset(offsets[0] / 8);
mpu.setYAccelOffset(offsets[1] / 8);
mpu.setZAccelOffset(offsets[2] / 8);
mpu.setXGyroOffset(offsets[3] / 4);
mpu.setYGyroOffset(offsets[4] / 4);
mpu.setZGyroOffset(offsets[5] / 4);
delay(2);
}
```



```
// пересчитываем для хранения
for (byte i = 0; i < 6; i++) {
    if (i < 3) offsets[i] /= 8;
    else offsets[i] /= 4;
}

// запись в память
EEPROM.put(START_BYTE, offsets);
}
```

↓ Мой алгоритм калибровки

```
// ===== ФУНКЦИЯ КАЛИБРОВКИ =====
void calibration() {
    long offsets[6];
    long offsetsOld[6];
    int16_t mpuGet[6];

    // используем стандартную точность
    mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);
    mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_250);

    // обнуляем оффсеты
    mpu.setXAccelOffset(0);
    mpu.setYAccelOffset(0);
    mpu.setZAccelOffset(0);
    mpu.setXGyroOffset(0);
    mpu.setYGyroOffset(0);
    mpu.setZGyroOffset(0);
    delay(10);

    for (byte n = 0; n < 10; n++) {        // 10 итераций калибровки
        for (byte j = 0; j < 6; j++) {    // обнуляем калибровочный массив
            offsets[j] = 0;
        }

        for (byte i = 0; i < 100 + BUFFER_SIZE; i++) {
            // делаем BUFFER_SIZE измерений для усреднения
            mpu.getMotion6(&mpuGet[0], &mpuGet[1], &mpuGet[2], &mpuGet[3], &mpuGet[4])
        }
    }
}
```



```

// пропускаем первые 99 измерений
if (i >= 99) {
    for (byte j = 0; j < 6; j++) {
        offsets[j] += (long)mpuGet[j];    // записываем в калибровочный массив
    }
}

for (byte i = 0; i < 6; i++) {
    offsets[i] = offsetsOld[i] - ((long)offsets[i] / BUFFER_SIZE); // учитыва
    if (i == 2) offsets[i] += 16384;                                // если ос
    offsetsOld[i] = offsets[i];
}

// ставим новые оффсеты
mpu.setXAccelOffset(offsets[0] / 8);
mpu.setYAccelOffset(offsets[1] / 8);
mpu.setZAccelOffset(offsets[2] / 8);
mpu.setXGyroOffset(offsets[3] / 4);
mpu.setYGyroOffset(offsets[4] / 4);
mpu.setZGyroOffset(offsets[5] / 4);
delay(2);
}
}

```

Также в библиотеке есть готовые функции для калибровки акселя и гироскопа, они принимают количество итераций калибровки (до 15):

```

mpu.CalibrateAccel(6);
mpu.CalibrateGyro(6);

```

Функции блокирующие, выполняются и автоматически выставляют оффсеты, т.е. датчик сразу калибруется. Для чтения оффсетов (например для записи в EEPROM) можно воспользоваться тем же способом, что и раньше:

```

mpu.getAXcelOffset();
mpu.getAYcelOffset();
mpu.getAZcelOffset();
mpu.getXGyroOffset();
mpu.getYGyroOffset();
mpu.getZGyroOffset();

```

Получение углов

Самый простой способ – получение углов чисто из ускорения через арксинус/арккосинус. Этот способ плох тем, что даёт угол в диапазоне ± 90 градусов, так как имеет место быть gimbal lock.

↓ Углы через тригонометрию

```
#include "MPU6050.h"
MPU6050 mpu;

void setup() {
  Wire.begin();
  Serial.begin(9600);
  mpu.initialize();    // запускаем датчик
}

void loop() {
  int16_t ax = mpu.getAccelerationX(); // ускорение по оси X
  // стандартный диапазон: +-2g
  ax = constrain(ax, -16384, 16384);    // ограничиваем +-1g
  float angle = ax / 16384.0;           // переводим в +-1.0

  // и в угол через арксинус
  if (angle < 0) angle = 90 - degrees(acos(angle));
  else angle = degrees(acos(-angle)) - 90;

  Serial.println(angle);
  delay(5);
}
```

Также акселерометр шумит и показания плавают, не говоря о минимальном разрешении $\pm 2g$ и резком возникновении ошибки при ускорении модуля вдоль измеряемой оси, то есть угол покоящегося модуля он покажет, но стоит начать его двигать – угол изменится. Для точного определения угла использую показания ускорения в связке с угловой скоростью.

Для получения углов из показаний акселерометра и гироскопа есть три основных способа:

- Фильтрация при помощи фильтра Калмана.
- Фильтрация при помощи комплементарного фильтра.



определение углов при помощи встроенного в датчик DMP (Digital Motion Processor).

Из всех трёх самый хороший результат даёт встроенный процессор, на его фоне Калман и комплементарии можно даже не рассматривать, они практически не работают. Пример рассмотрим в следующей главе ниже.

Ниже привожу совмещённый пример с упрощённым комплементарием, обычным и Калманом, всё в одном скетче. Но повторяю, по сравнению с DMP оно вообще не то показывает =)

↓ Скетч с фильтрами

```
#define COMPL_K 0.07
#include "Wire.h"
#include "Kalman.h"
Kalman kalmanX;
Kalman kalmanY;
uint8_t IMUAddress = 0x68;
/* IMU Data */
int16_t accX;
int16_t accY;
int16_t accZ;
int16_t tempRaw;
int16_t gyroX;
int16_t gyroY;
int16_t gyroZ;
float accXangle; // Angle calculate using the accelerometer
float accYangle;
float temp;
float gyroXangle = 180; // Angle calculate using the gyro
float gyroYangle = 180;
float compAngleX = 180; // Calculate the angle using a Kalman filter
float compAngleY = 180;
float kalAngleX; // Calculate the angle using a Kalman filter
float kalAngleY;
uint32_t timer;
void setup() {
  Serial.begin(9600);
  Wire.begin();
  Wire.beginTransmission(IMUAddress);
  Wire.write(0x6B); // PWR_MGMT_1 register
  Wire.write(0);    // set to zero (wakes up the MPU-6050)
  Wire.endTransmission(true);
  kalmanX.setAngle(180); // Set starting angle
  kalmanY.setAngle(180);
  timer = micros();
```

```

}
void loop() {
    //uint32_t looptime = micros();
    getValues();
    calculateAngles();
    //Serial.println(micros() - looptime);
    Serial.print(kalAngleX); Serial.print(" ");
    Serial.print(kalAngleY); Serial.println();
    /*
        Serial.print(compAngleX); Serial.print(" ");
        Serial.print(compAngleY); Serial.println();
    */
    delay(20); // The accelerometer's maximum samples rate is 1kHz
}

void getValues() {
    /* Update all the values */
    Wire.beginTransaction(IMUAddress);
    Wire.write(0x3B); // starting with register 0x3B (ACCEL_XOUT_H)
    Wire.endTransmission(false);
    Wire.requestFrom(IMUAddress, 14, true); // request a total of 14 registers
    accX = Wire.read() << 8 | Wire.read(); // 0x3B (ACCEL_XOUT_H) & 0x3C (ACCEL_XOUT_L)
    accY = Wire.read() << 8 | Wire.read(); // 0x3D (ACCEL_YOUT_H) & 0x3E (ACCEL_YOUT_L)
    accZ = Wire.read() << 8 | Wire.read(); // 0x3F (ACCEL_ZOUT_H) & 0x40 (ACCEL_ZOUT_L)
    tempRaw = Wire.read() << 8 | Wire.read(); // 0x41 (TEMP_OUT_H) & 0x42 (TEMP_OUT_L)
    gyroX = Wire.read() << 8 | Wire.read(); // 0x43 (GYRO_XOUT_H) & 0x44 (GYRO_XOUT_L)
    gyroY = Wire.read() << 8 | Wire.read(); // 0x45 (GYRO_YOUT_H) & 0x46 (GYRO_YOUT_L)
    gyroZ = Wire.read() << 8 | Wire.read(); // 0x47 (GYRO_ZOUT_H) & 0x48 (GYRO_ZOUT_L)
    //temp = ((float)tempRaw + 12412.0) / 340.0;
}

void calculateAngles() {
    /* Calculate the angles based on the different sensors and algorithm */
    accYangle = (atan2(accX, accZ) + PI) * RAD_TO_DEG;
    accXangle = (atan2(accY, accZ) + PI) * RAD_TO_DEG;
    float gyroXrate = (float)gyroX / 131.0;
    float gyroYrate = -((float)gyroY / 131.0);
    // Calculate gyro angle without any filter
    gyroXangle += gyroXrate * ((float)(micros() - timer) / 1000000);
    gyroYangle += gyroYrate * ((float)(micros() - timer) / 1000000);
    /*
        // Calculate gyro angle using the unbiased rate
        gyroXangle += kalmanX.getRate() * ((float)(micros() - timer) / 1000000);
        gyroYangle += kalmanY.getRate() * ((float)(micros() - timer) / 1000000);
    */
}

```

```

/*
// Calculate the angle using a Complimentary filter
compAngleX = ((float)(1 - COMPL_K) * (compAngleX + (gyroXrate * (float)(micros() - timer)
compAngleY = ((float)(1 - COMPL_K) * (compAngleY + (gyroYrate * (float)(micros() - timer)
*/

// Calculate the angle using a Kalman filter
kalAngleX = kalmanX.getAngle(accXangle, gyroXrate, (float)(micros() - timer)
kalAngleY = kalmanY.getAngle(accYangle, gyroYrate, (float)(micros() - timer)
timer = micros();
}

```

↓ Библиотека Kalman.h, положи рядом со скетчем

```

/* Copyright (C) 2012 Kristian Lauszus, TKJ Electronics. All rights reserved.

```

This software may be distributed and modified under the terms of the GNU General Public License version 2 (GPL2) as published by the Free Software Foundation and appearing in the file GPL2.TXT included in the packaging of this file. Please note that GPL2 Section 2[b] requires that all works based on this software must also be made publicly available under the terms of the GPL2 ("Copyleft").

Contact information

Kristian Lauszus, TKJ Electronics

Web : <http://www.tkjelectronics.com>

e-mail : kristianl@tkjelectronics.com

```

*/

```

```

#ifndef _Kalman_h

```

```

#define _Kalman_h

```

```

class Kalman {

```

```

public:

```

```

    Kalman() {

```

```

        /* We will set the variables like so, these can also be tuned by the use

```

```

        Q_angle = 0.001;

```

```

        Q_bias = 0.003;

```

```

        R_measure = 0.03;

```




```

    bias = 0; // Reset bias
    P[0][0] = 0; // Since we assume tha the bias is 0 and we know the start
    P[0][1] = 0;
    P[1][0] = 0;
    P[1][1] = 0;
};

// The angle should be in degrees and the rate should be in degrees per second
double getAngle(double newAngle, double newRate, double dt) {
    // KasBot V2 - Kalman filter module - http://www.x-firm.com/?page_id=
    // Modified by Kristian Lauszus
    // See my blog post for more information: http://blog.tkjelectronics.dk/

    // Discrete Kalman filter time update equations - Time Update ("Predict")
    // Update xhat - Project the state ahead
    /* Step 1 */
    rate = newRate - bias;
    angle += dt * rate;

    // Update estimation error covariance - Project the error covariance ahead
    /* Step 2 */
    P[0][0] += dt * (dt * P[1][1] - P[0][1] - P[1][0] + Q_angle);
    P[0][1] -= dt * P[1][1];
    P[1][0] -= dt * P[1][1];
    P[1][1] += Q_bias * dt;

    // Discrete Kalman filter measurement update equations - Measurement Update
    // Calculate Kalman gain - Compute the Kalman gain
    /* Step 4 */
    S = P[0][0] + R_measure;
    /* Step 5 */
    K[0] = P[0][0] / S;
    K[1] = P[1][0] / S;

    // Calculate angle and bias - Update estimate with measurement zk (newAngle)
    /* Step 3 */
    y = newAngle - angle;
    /* Step 6 */
    angle += K[0] * y;
    bias += K[1] * y;

    // Calculate estimation error covariance - Update the error covariance
    /* Step 7 */
    P[0][0] -= K[0] * P[0][0];

```



```

    P[0][1] -= K[0] * P[0][1];
    P[1][0] -= K[1] * P[0][0];
    P[1][1] -= K[1] * P[0][1];

    return angle;
};

void setAngle(double newAngle) { angle = newAngle; }; // Used to set angle,
double getRate() { return rate; }; // Return the unbiased rate

/* These are used to tune the Kalman filter */
void setQangle(double newQ_angle) { Q_angle = newQ_angle; };
void setQbias(double newQ_bias) { Q_bias = newQ_bias; };
void setRmeasure(double newR_measure) { R_measure = newR_measure; };

private:
/* Kalman filter variables */
double Q_angle; // Process noise variance for the accelerometer
double Q_bias; // Process noise variance for the gyro bias
double R_measure; // Measurement noise variance - this is actually the vari

double angle; // The angle calculated by the Kalman filter - part of the 2x
double bias; // The gyro bias calculated by the Kalman filter - part of the
double rate; // Unbiased rate calculated from the rate and the calculated b

double P[2][2]; // Error covariance matrix - This is a 2x2 matrix
double K[2]; // Kalman gain - This is a 2x1 matrix
double y; // Angle difference - 1x1 matrix
double S; // Estimate error - 1x1 matrix
};
#endif

```

Большую кучу теории про Эйлера и кватернионы можно почитать [здесь](#).

Оптимальное получение углов

Пример получения углов при помощи DMP и встроенной библиотеки

MPU6050_6Axis_MotionApps20.h. Единственное, скетч занимает прилично памяти (11кб Flash и 500б SRAM), так как фактически содержит прошивку для DMP и загружает её в модуль при каждом старте программы. С модуля можно подключить пин INT на один из пинов прерывания M'  имер на D2 в случае с Arduino Nano (номер прерывания 0). DMP будет дёргать

прерывание, сообщая о готовности, чтобы мы не тратили лишнее время на его постоянный опрос. Но опрашивать то можно и по таймеру (второй пример).

↓ Получаем углы с DMP по прерыванию (обновил скетч!)

```
#include "I2Cdev.h"
#include "MPU6050_6Axis_MotionApps20.h"
MPU6050 mpu;

volatile bool mpuFlag = false; // флаг прерывания готовности
uint8_t fifoBuffer[45];        // буфер

void setup() {
  Serial.begin(115200);
  Wire.begin();
  //Wire.setClock(1000000UL);    // разгоняем шину на максимум

  // инициализация DMP и прерывания
  mpu.initialize();
  mpu.dmpInitialize();
  mpu.setDMPEnabled(true);
  attachInterrupt(0, dmpReady, RISING);
}

// прерывание готовности. Поднимаем флаг
void dmpReady() {
  mpuFlag = true;
}

void loop() {
  // по флагу прерывания и готовности DMP
  if (mpuFlag && mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
    // переменные для расчёта (ypr можно вынести в глобал)
    Quaternion q;
    VectorFloat gravity;
    float ypr[3];

    // расчёты
    mpu.dmpGetQuaternion(&q, fifoBuffer);
    mpu.dmpGetGravity(&gravity, &q);
    mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

    mpuFlag = false;
```



```
// выводим результат в радианах (-3.14, 3.14)
Serial.print(ypr[0]); // вокруг оси Z
Serial.print(',');
Serial.print(ypr[1]); // вокруг оси Y
Serial.print(',');
Serial.print(ypr[2]); // вокруг оси X
Serial.println();
// для градусов можно использовать degrees()
}
}
```

↓ Без прерываний, но с таймером

```
#include "I2Cdev.h"
#include "MPU6050_6Axis_MotionApps20.h"
MPU6050 mpu;

uint8_t fifoBuffer[45];          // буфер

void setup() {
    Serial.begin(115200);
    Wire.begin();
    //Wire.setClock(1000000UL);    // разгоняем шину на максимум

    // инициализация DMP
    mpu.initialize();
    mpu.dmpInitialize();
    mpu.setDMPEnabled(true);
}

void loop() {
    static uint32_t tmr;
    if (millis() - tmr >= 11) {    // таймер на 11 мс (на всякий случай)
        if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
            // переменные для расчёта (ypr можно вынести в глобал)
            Quaternion q;
            VectorFloat gravity;
            float ypr[3];

            // расчёты
            mpu.dmpGetQuaternion(&q, fifoBuffer);
```

```

mpu.dmpGetGravity(&gravity, &q);
mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

// выводим результат в радианах (-3.14, 3.14)
Serial.print(ypr[0]); // вокруг оси Z
Serial.print(',');
Serial.print(ypr[1]); // вокруг оси Y
Serial.print(',');
Serial.print(ypr[2]); // вокруг оси X
Serial.println();
// для градусов можно использовать degrees()

tmr = millis(); // сброс таймера
}
}
}

```

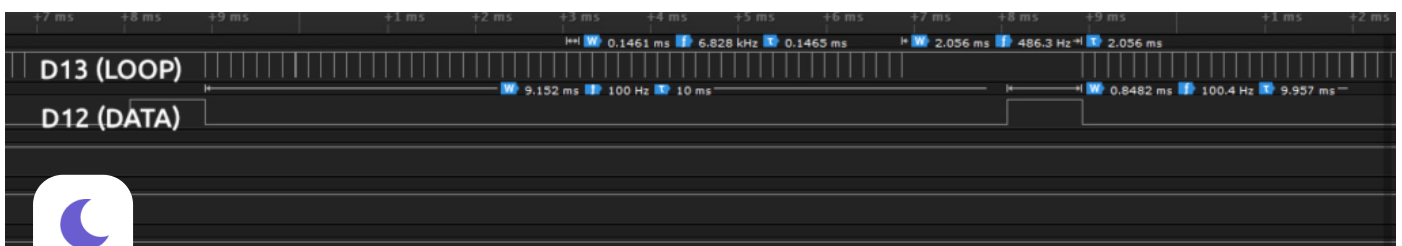
По производительности дела обстоят так: без прерываний нам приходится проверять готовность DMP, что занимает 146 мкс, а сам DMP готов отдавать данные через каждые 10 мс (9.1 + 0.8). Обработка данных занимает 2 мс, из которых половина – обработка буфера (функция `dmpGetCurrentFIFOpacket`). Тестовый код для отладки через анализатор выглядел так:

```

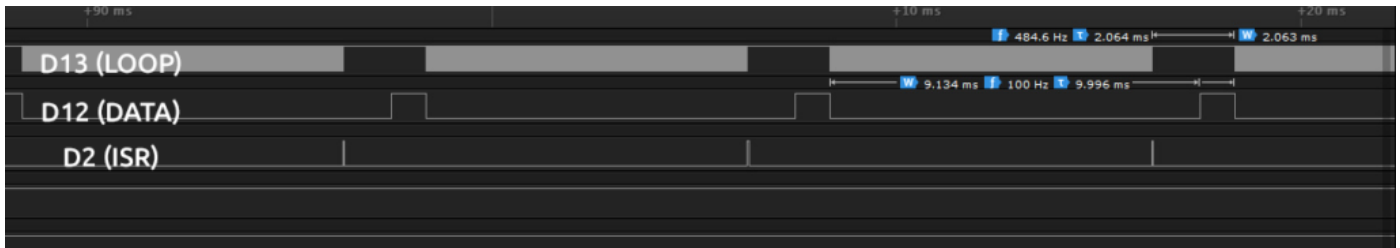
void loop() {
  fastWrite(13, 1);
  if (mpu.dmpGetCurrentFIFOpacket(fifoBuffer)) {
    fastWrite(12, 1);
    mpu.dmpGetQuaternion(&q, fifoBuffer);
    mpu.dmpGetGravity(&gravity, &q);
    mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
    fastWrite(12, 1);
  }
  fastWrite(13, 0);
}

```

И получились вот такие тайминги:



В реальном примере с таймером мы не тратим 146 мкс в цикле, а просто асинхронно ждём 10 (а лучше 11) миллисекунд. Работа с прерыванием или с таймером будет выглядеть более красиво:



Если вы вдруг ничего не поняли: оба примера можно смело использовать, прерывание – не панацея, если в программе есть возможность считать время и поднять программный таймер, как во втором примере. Но с прерыванием можно безо всяких таймеров точно поймать момент, когда DMP обработал данные. Если это вдруг нужно.

Некоторые примеры

↓ Определение периода тряски вдоль оси X

Скетч временно не работает, надеюсь починю

```
// скетч определения периода тряски акселерометра
#define DEADZONE 4000
#define MAX_LOOP 900
#include "Wire.h"
#include "I2Cdev.h"
#include "MPU6050.h"
MPU6050 mpu;

float k = 0.1;
int16_t ax, ay, az, ax_f;
int16_t gx, gy, gz;
int16_t loopTime;
boolean toggle_flag;
unsigned long maxTimer, analyzeTimer;

void setup() {
  Wire.begin();
  Serial.begin(9600);
  mpu.initialize();
  Serial.println(mpu.testConnection() ? "MPU6050 connection successful" : "MPU6050 connection failed");

  // Acceleration range: ± 2   ± 4   ± 8   ± 16 g
  // 1G value:           16384 8192 4096 2048
  // MAX G value:        32768
}
```

```

mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_8);

// Gyroscope range: 250  500  1000 2000 °/s
// MAX value:          32768
mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_250);

pinMode(13, OUTPUT);
}

void loop() {
    if (millis() - analyzeTimer >= 2) { // шаг анализа периода
        mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz); // запросить данные
        ax_f += (ax - ax_f) * 0.1; // отфильтровать
        if (ax_f > DEADZONE && toggle_flag) {
            toggle_flag = false;
            digitalWrite(13, 1);
            loopTime = millis() - maxTimer; // расчёт периода движения
            maxTimer = millis();
        } else if (ax_f < -DEADZONE && !toggle_flag) {
            toggle_flag = true;
            digitalWrite(13, 0);
            loopTime = millis() - maxTimer;
            // расчёт периода движения
            maxTimer = millis();
        }
        if (millis() - maxTimer > MAX_LOOP) loopTime = 0;
    }
    Serial.println(loopTime);
    analyzeTimer = millis();
}

```

↓ Определение периода вращения (ось Y по радиусу)

```

// скетч определения периода вращения акселерометра
#define DEADZONE 700
#define MIN_LOOP 100
#define MAX_LOOP 900

#include "Wire.h"
#include "I2Cdev.h"

```

```
#include "MPU6050.h"

MPU6050 mpu;
float k = 0.2;
int16_t ax, ay, az, ay_f;
int16_t gx, gy, gz;
int last_acc, level;
boolean rising_flag = true, falling_flag = false, toggle_flag;
unsigned long maxTimer, analyzeTimer;
int loopTime;

void setup() {
  Wire.begin();
  Serial.begin(9600);
  mpu.initialize();
  Serial.println(mpu.testConnection() ? "MPU6050 connection successful" : "MPU6050 connection failed");

  // Acceleration range: ± 2   ± 4   ± 8   ± 16 g
  // 1G value:           16384 8192 4096 2048
  // MAX G value:       32768
  mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_16);

  // Gyroscope range: 250   500  1000 2000 °/s
  // MAX value:       32768
  mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_250);
}

void loop() {
  if (millis() - analyzeTimer > 3) { // шаг анализа периода
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz); // запросить данные
    ay_f += (ay - ay_f) * k; // отфильтровать
    if (rising_flag) // поднимаемся
      // ждём падения сигнала на величину DEADZONE
      if (last_acc < ay_f + DEADZONE) {
        // если сигнал не упал, принять величину как точку для следующего сравнения
        last_acc = ay_f;
        toggle_flag = true;
      } else {
        // если есть падение, и прошёл минимальный период
        // (защита от всяких случайных ударов и дёрганий)
        if (toggle_flag && (millis() - maxTimer > MIN_LOOP)) {
          toggle_flag = false;
          rising_flag = false;
        }
      }
    }
  }
}
```




```

        falling_flag = true;
        loopTime = millis() - maxTimer;    // расчёт периода движения
        maxTimer = millis();
        //level = last_acc;
    }
}

if (falling_flag) {    // опускается
    // ждём прироста сигнала на величину DEADZONE
    if (last_acc > ay_f - DEADZONE) {    // падаем
        // если сигнал не вырос, принять величину как точку для следующего сравн
        last_acc = ay_f;
        toggle_flag = true;
    } else {
        // если есть прирост, и прошёл минимальный период
        // (защита от всяких случайных ударов и дёрганий)
        if (toggle_flag && (millis() - maxTimer > MIN_LOOP)) {
            toggle_flag = false;
            rising_flag = true;
            falling_flag = false;
            //level = last_acc;
        }
    }
}

/*
    Serial.print("$");
    Serial.print(ay_f);
    Serial.print(" ");
    Serial.print(level);
    Serial.print(" ");
    Serial.print(loopTime*10);
    Serial.println(";");
*/

if (millis() - maxTimer > MAX_LOOP) loopTime = 0;
Serial.println(loopTime);
analyzeTimer = millis();
}
}

```

```
// простая сигнализация
// датчик реагирует даже на постукивание
// пальцем на противоположном конце стола
#include "Wire.h"
#include "I2Cdev.h"
#include "MPU6050.h"

MPU6050 mpu;
int16_t ax, ay, az;
int16_t gx, gy, gz;
long ACC, GYR;
long maxACC, maxGYR, trACC, trGYR;

void setup() {
  Wire.begin();
  Serial.begin(9600);
  mpu.initialize();
  mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);
  delay(2000);

  // "калибровка" максимальных значений
  for (int i = 0; i < 30; i++) {
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    ACC = abs(ax) + abs(ay) + abs(az);
    GYR = abs(gx) + abs(gy) + abs(gz);
    if (ACC > maxACC) maxACC = ACC;
    if (GYR > maxGYR) maxGYR = GYR;
    delay(5);
  }
  maxACC = maxACC + trACC;
  maxGYR = maxGYR + trGYR;
}

void loop() {
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  // сумма модулей
  ACC = abs(ax) + abs(ay) + abs(az);
  GYR = abs(gx) + abs(gy) + abs(gz);
  if (ACC > maxACC || GYR > maxGYR) {
    Serial.println("KEK!");
    delay(10);
  }
}
```



```
}
```

↓ Прерывание по движению (NEW!)

Настраиваем датчик на выдачу прерываний по движению (например для сигнализации). Таким же образом можно настроить на прерывания по свободному падению, см. документацию выше.

```
#include "MPU6050.h"
MPU6050 mpu;

void setup() {
    // запускаем всё
    Wire.begin();
    Serial.begin(9600);
    mpu.initialize();

    mpu.setInterruptMode(true);           // включаем режим прерываний
    mpu.setIntMotionEnabled(true);        // включаем прерывания движения
    mpu.setMotionDetectionThreshold(2);   // порог 0..255
    mpu.setMotionDetectionDuration(2);    // таймаут 0..255
    attachInterrupt(0, isr, RISING);      // прерывание
}

volatile bool motion;
void isr() {
    motion = true; // флаг прерывания
}

void loop() {
    // по флагу прерывания пишем в порт
    // прерыванием можно будить МК и так далее
    if (motion) {
        Serial.println("Motion!");
        motion = false;
    }
}
```

Что касается получения линейных скоростей и перемещений при помощи двойного интегрирования ускорения (инерционная навигация), то задача не очень простая, нужны



хитрые многоосевые фильтры и желательно процессор помощнее, также об этом можно почитать [вот тут](#).

4.9/5 - (34 голоса)

ПОХОЖИЕ ЗАПИСИ

Работа с Arduino и RFID
MFRC522

Строим графики на LCD
дисплеях 1602 и 2004

Все математические функции
Arduino

Гайд
светоди

✉ Подписаться ▼



Присоединиться к обсуждению

B *I* U

70 КОММЕНТАРИЕВ



Старые ▼



Гость ⌚ 2 лет назад

Здравствуйте, сделал по инструкциям в интернете мышь на базе три 6050, но как сделать, чтобы курсор мыши при первом нажатии останавливался, а при повторном продолжал двигаться с того же места, т.е. чтобы устройство работало как аэропульт для смарт-приставок?

Сейчас при нажатии курсор останавливается. И если отклонить устройство и отпустить кнопку, то курсор начинает движение с нового места.

Последний раз редактировалось 2 лет назад Гость ем

^ 0 v ➡ Ответить

Просмотр ответов (2) ▼



sknek ⌚ 2 лет назад

Спасибо за отличную статью! У меня непонимание по поводу калибровки. Почему такой сложный алгоритм? Что если с помощью функции `dmpGetYawPitchRoll` замерить средние углы, которые выдаёт MPU в спокойном состоянии, и затем просто вычитать их в дальнейшем?

0 Ответить

Просмотр ответов (3)



Лёха 2 лет назад

Скетч “Углы через DMP”, похоже не рабочий, выводит 0 тупо. Я так надеялся на него. Родной пример из бибы не взлазит в 168 атмегу... Остальные работают нормально.

0 Ответить

Просмотр ответов (1)



mirok 2 лет назад

Пытаюсь повторить определитель вибрации, получаю при компиляции ошибку:

vibrocontrol:28:21: error: ‘trACC’ was not declared in this scope

```
maxACC = maxACC + trACC;
```

Подскажите, что делать и кто виноват?

0 Ответить

Просмотр ответов (2)



Антон 2 лет назад

здравствуйте, комунибудь можно объяснить как получить это?

- Ускорение в м/с/с при чувствительности 2: `float accX_f = accX / 32768 * 2`

0 Ответить

Просмотр ответов (1)



flipflop 2 лет назад

Калибровка оффсетов 2 раза напечатана)

1 Ответить



Tarik 2 лет назад

Получение углов с DMP по прерыванию, это просто бомба, ти просто мой кумир и я тебя люблю)))

Только тут исправьте не все гуманитарии поймут что надо смотреть предыдущий код чтобы понять где ошибка.

Получаем углы с DMP по прерыванию

```
#include
```

```
#include
```

```
MPU6050 mpu;
```



0 Ответить

Просмотр ответов (1)

**Гость** 2 лет назад

спасибо за развернутую статью 😊
и если сайт так корёжит код, может, его (код) архивом выложить?

✎ Последний раз редактировалось 2 лет назад Гость ем

^ 0 v Ответить

Просмотр ответов (1) v

**Tarik** 2 лет назад

Алекс помощи пожалуйста, на основе этого датчика и скетча **Получаем углы с DMP по прерыванию** я хочу сделать самодельный контроллер автомат pp gun controller (будешь иметь тему для классного видео на канале) Я плохой программист и у меня не получается написать код для вычисления скорости перемещения этого датчика, не все так просто если вдуматься вся проблема кроется в этих числах -3.14 – 3.14. И так главный вопрос, как мне сбросить датчик на 0.00 по окончании собственного кода который я допишу к скетчу, таким нехитрым способом Я бы смог вычислить скорость перемещения в секунду. Надеюсь что там есть такой метод класса.

^ 0 v Ответить

Просмотр ответов (4) v

**Александр** 2 лет назад

Привет Алекс, у тебя в коде : **“Мой вариант калибровки”** в самом конце лишняя фигурная скобка “{“

^ 3 v Ответить

**Георгий** 2 лет назад

Красава, большое спасибо тебе

^ 0 v Ответить

**Николай** 2 лет назад

Что то не вкурил насчет DMP функций. Читаю даташит map registers для MPU и там вообще нет регистров и битов для DMP. Пытаюсь залезать в ардуиновские библиотеки, нахожу функции для DMP но они ссылаются на биты в регистрах, которых reserved. Ну может я не ту спецификацию читаю, но даже в обычном даташите на dmp выделен один маленький абзац, как будто это какая то мелкая функций, поэтому я в ступоре. Просто сейчас стоит выбор, что покупать MPU6050 или BMI160. В комментариях под заказами пишут что BMI160 точнее, чем MPU6050, но в BMI160 я не увидел ничего похожего на DMP. В принципе, можно попробовать купить BMI160 и использовать программные фильтры, например фильтр Маджвика. Пишут что он даже точнее Калмана, тем более мне нужно не для квадрокоптера. Но меня привлек DMP потому что это разгрузило бы главный процессор, а так приходится считать все в коде, не очень хорошо. Ну у меня в принципе не arduino а STM с FPU и бОльшей тактовой частотой, но все равно функция привлекательная. Короче, может кто меня просветит насчет этого DMP ибо это какая то залипуха. В даташите с регистрами нет даже



регистров хотя бы как то относящихся в DMP. Возможно у автора поста обычные фильтры не работали, потому что он использует arduino где числа с плавающей запятой вычисляются программно а не аппаратно. Короче если DMP незадокументирован, то такой чип покупать наверное не стоит.

^ 0 ✓ ➔ Ответить

Просмотр ответов (1) ▼



Aliaksandr ⌚ 1 год назад

Добрый день хочу собрать перчатку стробоскоп. Но не получается: через несколько импульсов на светодиоде проходит задержка на 1 секунду примерно. Потом все повторяется снова. К тому же полностью напрочь не работает акселерометр. Подскажите что проверить куда залезть. Код брал со страницы проекта Алекса “Перчатка стробоскоп” Вот тут я записал как оно неправильно работает.

<https://disk.yandex.ru/i/ZMDNsoOBw-alhw>

^ 0 ✓ ➔ Ответить



Дмитрий ⌚ 1 год назад

Доброго времени суток! Подскажите пожалуйста, как максимально снизить энергопотребление этого модуля, но при этом оставить рабочим прерывание по движению? Заранее спасибо 😊

^ 0 ✓ ➔ Ответить



Георгий ⌚ 1 год назад

Здравствуйте, можете подсказать как подключить 4 штуки MPU6050 и плюс добавить в схему zs-040 (блютус модуль)

^ 0 ▼ ➔ Ответить

Просмотр ответов (2) ▼



Georges ⌚ 1 год назад

метр/секунда/секунда = (метр*секунда)/секунда. Должно быть: (метр/секунда)/секунда

^ 1 ▼ ➔ Ответить

Просмотр ответов (2) ▼



Илья ⌚ 1 год назад

Добрый день! У меня два вопроса: 1. Странно, но ах, ау возвращают 32768 и -32767 соответственно, аз что-то в районе 1700. gx, gy, gz – вроде что-то осмысленное, но мне нужны именно статические углы поворота. Это у менядохлый датчик или я что-то делаю не так? 2. “has no member named ‘dmpGetCurrentFIFOPacket’; did you mean ‘dmpProcessFIFOPacket’?” – обновилась библиотека?

^ 0 ✓ ➔ Ответить



Николай ⌚ 1 год назад

Здравствуйте, можно ли получить значения по каждой из осей в диапазоне 0-360

 0   ОтветитьПросмотр ответов (3) **Tarik**  1 год назад

С esp32 будет работать, скажите умоляю.

 0   ОтветитьПросмотр ответов (2) **Алексей**  1 год назад

Добрый день! Столкнулся с проблемой., после отключения питания (Отсоединяю USB и подключаю обратно) перестает работать передача данных с MPU6050 в Serial port Arduino Nano, помогает только повторная заливка скетча, то есть сразу после заливки все работает., но стоит передернуть USB все перестает работать. Не подскажите в чем может быть проблема?

 0   Ответить**Антон**  1 год назад

.

 Последний раз редактировалось 1 год назад Антон ем 0   Ответить**Юрий**  1 год назад**Алекс, благодарю за статью!**

Есть такой проект: робот с качающейся вверх-вниз головой. Он едет только вперед. На голове должны стоять два датчика расстояния типа **SR04**, из них один смотрит +- по горизонтали перед собой, второй – под ноги (примерно 45 градусов к горизонту). Неизвестен начальный угол наклона головы, известна лишь её высота над полом. Посредством модуля **MPU6050** я планирую вычислять куда датчики смотрят (иногда датчик, обрабатывающий пол, может смотреть себе в ноги, а датчик, обрабатывающий пространство перед роботом – соответственно, в пол) и вычислять расстояние до пола. Суть всего проекта: находить ступеньки или ямки (ну, и также не врезаться в стены). В проекте приветствуется малогабаритность, поэтому главный вопрос: **потянет ли НАНО связку MPU6050+SR04+SR04** в плане тяжести программы? Если нет, то посоветуйте платформу, на которой стоит собирать. Благодарю за ответ, с уважением, ваш давний подписчик 😊

 0   Ответить**Юрий**  1 год назад

Алекс, подскажите как зафиксировать полученный угол? В примере “Получаем углы с DMP” при наклонах датчика в мониторе значения меняются в +-, но после остановки датчика в каком-либо положении в течение 5 секунд значения вновь приходят к нулю как после калибровки. Как я понял, без акселерации гироскоп перестаёт выдавать значения отличные от калибровки. Как можно решить этот вопрос?



 0   Ответить**Артём**  1 год назад

Здравствуй Александр! Пытаюсь разобраться с Raspberry pi 3 на C++ и MPU6050. Данные приходят но огромные и колеблющиеся. Скажи есть ли у тебя программа или ссылки на них, чтобы определить угол с помощью гироскопа и акселерометра? Возможно ссылки? Заранее огромное спасибо!

 0   Ответить**Александр**  1 год назад

Здравствуй, подскажите примером как это сделать пожалуйста "// для градусов можно использовать degrees()"

 0   Ответить**xx xxx**  1 год назад

Не получается залить в Uno код для определения углов с DMP, не важно, по таймеру(желательно) или по прерыванию. IDE пишет ошибку "Ошибка компиляции для платы Arduino Uno." Голову сломал, не знаю, куда воевать.

 0   ОтветитьПросмотр ответов (1) **Константин**  1 год назад

Помогите пожалуйста. Загрузил прошивку с DMP, значения преобразовал в градусы. и вроде все хорошо, угол определяет правильно, что важно – шустро(делаю стабилизацию). но стоит только по одной из осей наклонить на 90 градусов, как другая ось тут же становится 170 либо -170, хотя по другой оси никаких движений не было. В фильтре Калмана похожая история. может микросхема с браком? Просто что я мог сделать неправильно в прошивке с DMP

 0   ОтветитьПросмотр ответов (1) **Oleg**  1 год назад

```
float accX_f = accX / 32768 * 2
```

```
float accX_f = (float)accX / 32768 * 2
```

 Последний раз редактировалось 1 год назад Oleg ем 0   Ответить**Владимир**  1 год назад

Здравствуй, возник вопрос. Гироскоп нужно калибровать при каждом включении или достаточно откалибровать 1 раз и записать оффсеты в код?

 0   Ответить

**Сергей** 1 год назад

Здравствуйте, а вы прд заказ сможете написать скетч для акселерометра и гастроить его работу? Пишите сюда: 010103@inbox.ru

0 Ответить

**Medetkhan** 1 год назад

Здравствуйте! Спасибо за полезную статью! У меня вопрос, подходит ли эти коды для подключения других IMU например, MPU92/65? Если нет, как узнать где что менять в коде, чтобы адаптировать? Спасибо заранее!

0 Ответить

**Артем** 1 год назад

Здравствуйте, очень полезная статья, особенно про углы через DMP.

Скажите пожалуйста, можно ли избавиться от **смещения по оси Z**? Ошибка накапливается при включении, около 5 градусов, но больше всего **при повороте**, и именно в меньшую сторону. Изменял частоту опроса датчика, просто убирал таймер, не помогает, даже при медленном повороте.

В интернете ничего внятного не нашел, к сожалению.

В моём проекте поворот относительно оси Z это самое важное, и, если невозможно исправить дрейф на данном датчике, то какой можете посоветовать? Модуль-компас HMC5883L не подходит, рядом много сервоприводов.

0 Ответить

**Tarik** 1 год назад

```
if (millis() > tmr ) { // таймер на 11 мс  
  tmr = millis() + 11; // сброс таймера  
}
```

0 Ответить

**Юрий** 10 месяцев назад

Доброго времени суток.

Помимо комплиментарного и фильтра Калмана не рассматривали фильтры Махони и Маджвика?

И в чем именно выражается “повторюсь, по сравнению с DMP оно вообще не то показывает =)” ?)

Все ж DMP медленный. Но, к слову, и его по документации можно быстрее использовать

“Typically, motion processing algorithms should be run at a high rate, often around 200Hz..”, т.е. в 2 раза быстрее, чем у Вас в статье.

За ранее благодарю 😊



0 Ответить

Просмотр ответов (1)

**Сергей** 10 месяцев назад

Всем привет! У меня возникла проблема при работе с этим модулем. Я пытался к одной ардуине подключить модуль mrf42l01 и mpu6050. оба заводятся, но по факту работает только mpu, по отдельности они прекрасно работают. Кто-нибудь подскажет как это исправить?

^ 0 v Ответить

Просмотр ответов (1) v

**Farych** 7 месяцев назад

Здравствуйте, может ли акселерометр измерять виброскорость и виброускорение электродвигателя?

^ 0 v Ответить

**mlev** 3 месяцев назад

Все работает. Но первые ~15 сек после старта скетча каждый раз происходит подстройка (усреднение) параметров микросхемы (речь не про скетч калибровки). И если в эти первые 15 сек менять положение микросхемы, то подстройка параметров так и не заканчивается. По крайней мере до тех пор пока положение микросхемы не будет стабилизировано на 15 сек.

Подскажите, пожалуйста, как можно сократить время подстройки или отключить ее?

^ 0 v Ответить

**Анатолий** 2 месяцев назад

Просто шедеврально! Если есть пример измерения скорости и расстояния выложите пожалуйста! Вы супер!

^ 0 v Ответить

**AlexÖris** 1 месяц назад

Вот бы ктонибудь сделал устройстве стедикам для фото и видео съёмки на основе этого проекта, то ему бы были благодарны десятки тысяч фотографов и видеооператоров

^ 0 v Ответить

**Богдан** 1 месяц назад

Добрый день, большое спасибо за статью. У меня программа по определению угла по таймеру отлично работает на arduino uno, но нужно её запустить именно на esp32.

Когда загружаю туда скетч – вылазят ошибки. В интернете нашёл версию

MPU6050_6Axis_MotionApps20.h специально под esp32

(https://github.com/mwesterm/esp32-vario/blob/master/MPU6050_6Axis_MotionApps20.h),

но ошибки даже с ней всё-равно лезут Скажите, пожалуйста, как написать программу для определения угла наклона правильно? Спасибо



 0   Ответить**Planer** ⌚ 6 дней назад

Полезная инфа!

 0   Ответить[Arduino и датчик MPU6050](#)[Подключение](#)[Получение сырых данных](#)[Библиотека MPU6050 от i2cdev](#)[Расшифровка сырых данных](#)[Настройка чувствительности](#)[Калибровка оффсетов](#)[Алгоритмы калибровки](#)[Получение углов](#)[Оптимальное получение углов](#)[Некоторые примеры](#)[ПОХОЖИЕ ЗАПИСИ](#)

Copyright © 2023 AlexGyver Technologies

